Terraform Interview Prep — Q&A

Prepared for Ketan Dhadve — DevOps Engineer Intern

Q1. What is the difference between terraform plan and terraform apply, and how does Terraform know what changes need to be made?

terraform plan reads your .tf config files, reads the state file (terraform.tfstate), and queries the real infrastructure via the provider API to check current status. It compares desired state (config) against current state (state file + real infra), then shows a preview of what will be created, changed, or destroyed — nothing is touched yet.

terraform apply runs the same comparison internally, then actually executes the API calls to create/update/delete resources, and updates the state file to reflect the new reality.

Terraform decides what to change by comparing three things: your .tf files (desired state), the state file (last known state), and the real infrastructure (current actual state via provider refresh). From the diff, it builds a dependency graph and calculates the minimal set of actions needed, in the correct order.

Q2. What is the Terraform state file, and why is it important? What problems happen if two people run terraform apply at the same time, and how do you solve that?

terraform.tfstate is a JSON file where Terraform tracks every resource it manages — IDs, attributes, dependencies. It's Terraform's source of truth; without it, every apply would look like creating everything from scratch.

If two people run apply at the same time using local or unlocked shared state, both read the same starting state, calculate plans independently, and start making API calls. Whichever apply finishes last overwrites the other's state changes, causing drift — duplicate resources, orphaned resources, or a state file that no longer matches real infrastructure.

The fix is remote state plus state locking: store state in a versioned S3 bucket, and use a DynamoDB table for locking. When someone runs apply, Terraform writes a lock entry in DynamoDB; anyone else attempting to apply is blocked until the lock releases. (Newer Terraform versions also support native S3 locking without DynamoDB.)

Q3. What's the difference between a Terraform module, a provider, and a resource?

A resource is the basic building block — a single infrastructure object Terraform creates and manages, e.g. resource "aws_instance" "web" { ... }. Each resource block maps to one real object in the cloud.

A provider is the plugin that lets Terraform talk to a specific platform's API (AWS, Azure, GCP, Kubernetes, GitHub, etc.) and translates .tf code into real API calls, e.g. provider "aws" { region = "ap-northeast-1" }.

A module is a reusable, packaged group of resources — a folder of .tf files that can be called multiple times with different inputs instead of duplicating code. Every Terraform project has a root module, and you can also call child modules (your own, or from the public Terraform Registry). Example: wrapping VPC or EKS cluster creation as a module so the same code can be reused across dev/staging/prod by changing input variables.

Q4. What is the purpose of terraform init, and what does it actually set up? What's the difference between terraform destroy and manually deleting resources from the AWS console?

terraform init is the first command run in any new or freshly cloned Terraform project. It downloads provider plugins declared in the provider/required_providers blocks, initializes the backend (where state is stored — local file or remote like S3), downloads any referenced modules into a .terraform folder, and creates a .terraform.lock.hcl file that pins provider versions for the whole team. It needs to be re-run whenever a provider, module source, or backend config changes.

terraform destroy reads the state file, determines everything Terraform currently manages, and deletes those resources in the correct dependency order, then updates the state file to reflect that everything is gone.

Manually deleting a resource from the AWS console removes it in reality, but Terraform's state file doesn't know that — this creates drift, where state no longer matches actual infrastructure. The next plan will either try to recreate the resource or error out because something it depends on is missing. Drift is detected with plan and resolved with either apply (to restore it per config) or terraform import (to bring an out-of-band resource back under management).

Q5. What is terraform.tfvars, and what's the difference between input variables, output values, and locals?

Input variables (variable block) are the parameters of a Terraform configuration — values passed in from outside so the same code can be reused across environments, e.g. variable "instance_type" { type = string }, referenced in code as var.instance_type.

terraform.tfvars is a file that assigns actual values to those variables instead of hardcoding them, e.g. instance_type = "t3.medium". Terraform auto-loads terraform.tfvars or *.auto.tfvars, and separate files like dev.tfvars or prod.tfvars can be passed explicitly with -var-file for different environments.

Locals (locals block) are internal, computed values used to avoid repeating an expression or to improve readability — not configurable from outside, referenced as local.name.

Output values (output block) are values Terraform returns after apply, useful for displaying important info (like a resource endpoint) or passing data between modules, where a child module's output becomes a parent module's input.

Q6. What is the Terraform dependency graph, and how does Terraform decide the order to create/destroy resources? What's the difference between implicit and explicit dependencies (depends_on)?

Before creating anything, Terraform builds a directed acyclic graph (DAG) mapping which resources depend on which others. It walks this graph to decide execution order, running independent resources in parallel and dependent resources strictly in sequence.

Implicit dependencies are the most common — Terraform infers them automatically whenever one resource references another resource's attribute, e.g. a subnet referencing aws_vpc.main.id automatically tells Terraform to create the VPC first.

Explicit dependencies (depends_on) are used when two resources are logically related but there's no direct attribute reference in code for Terraform to detect — for example, an EKS node group needing an IAM role to be fully attached before it can authenticate, even though no attribute is directly referenced.

Q7. What's the difference between terraform apply -replace (or the older terraform taint) and just deleting a resource block from your .tf file?

terraform apply -replace (previously terraform taint) tells Terraform to destroy and recreate one specific resource on the next apply, while leaving it in your configuration and leaving every other resource untouched. It's used when a resource is broken or misconfigured outside of Terraform's knowledge and needs a clean rebuild.

Deleting the resource block from your .tf file instead tells Terraform the resource should no longer exist at all — on the next apply, Terraform will destroy it and remove it from state permanently, with no recreation.

The key difference: -replace recreates a resource you still want; removing the block eliminates a resource you no longer want.

Q8. What is a Terraform workspace, and how is it different from using separate .tfvars files per environment?

A Terraform workspace lets you maintain multiple, isolated state files from the same configuration directory — e.g. terraform workspace new dev creates a separate state file for dev without duplicating .tf code.

Using separate .tfvars files (dev.tfvars, prod.tfvars) with -var-file keeps a single shared state but changes input values per run.

The practical difference: workspaces isolate state (safer for accidentally applying to the wrong environment) but share the exact same code and backend config; separate .tfvars files change values but by default still point at the same state unless combined with separate backend configs. Many teams prefer fully separate directories or backend configs per environment over workspaces for production use, since workspaces can make it easy to forget which environment you're currently targeting.

Q9. What does count do in a resource block, and what's the difference between count and for_each? Why prefer one over the other?

count lets you create multiple instances of a resource by specifying a number, e.g. count = 3 creates three copies, addressed by index (resource.name[0], resource.name[1], etc.).

for_each creates multiple instances from a map or set of strings, addressed by key rather than index (resource.name["key"]).

The key difference matters when the list changes: with count, removing an item from the middle of a list shifts every subsequent index, causing Terraform to destroy and recreate resources that didn't actually change. With for_each, each instance is tied to a stable key, so removing one item only affects that specific instance. for_each is generally preferred when instances are meaningfully distinct (e.g. one per named environment or named IAM user); count is fine for simple, truly identical repeated resources.

Q10. What is terraform import, and when would you need to use it?

terraform import brings an existing piece of infrastructure — one that was created manually, outside of Terraform, or whose state was lost — under Terraform's management by linking it to a resource block in your state file, without recreating it.

It's needed whenever real infrastructure and Terraform's state have drifted apart: for example, if an IAM role or EKS access entry was created manually via the AWS CLI or console (similar to the access entry created via CloudShell in the flightsystem-1 EKS troubleshooting), import lets Terraform start tracking that resource going forward instead of trying to recreate a duplicate or erroring out because the resource already exists.

After importing, you still need a matching resource block written in your .tf config; import only populates the state file — it does not generate configuration for you (though newer Terraform versions support generating config automatically alongside import).

Appendix — Deep Dive: Most Frequently Asked Topics

A1. Terraform State — Why It Exists and How to Manage It Safely in a Team

Terraform maps declarative config to real infrastructure. To do this safely on repeat runs, it must remember what it already created — including internal IDs that never appear in your code (e.g. an EC2 instance ID assigned by AWS at creation time). This is stored in terraform.tfstate.

State is used for: (1) mapping config resources to real-world objects via IDs, (2) storing attributes so plan doesn't need to query everything from the provider API every run, (3) tracking dependencies for the execution graph, and (4) enabling collaboration — a shared state file is how a team agrees on current reality.

Problem with local state in a team: nobody else gets accurate plans, concurrent applies can silently create duplicate/conflicting resources, and losing the laptop means losing Terraform's only record of what it manages (the infra still exists, but Terraform 'forgets' it).

Solution — remote state + locking:

```
terraform {
  backend "s3" {
    bucket         = "my-company-terraform-state"
    key            = "flightsystem/eks/terraform.tfstate"
    region         = "ap-northeast-1"
    dynamodb_table = "terraform-locks"
    encrypt        = true
  }
}
```

S3 bucket stores state centrally with versioning enabled (roll back a corrupted state). encrypt = true protects secrets that often live in state in plaintext. The DynamoDB table provides state locking — Terraform writes a lock item before an apply; a concurrent apply attempt is blocked with 'Error acquiring the state lock' until the lock releases.

Useful state commands: terraform state list (see tracked resources), terraform state show <resource> (inspect attributes), terraform state mv (rename/move without destroy+recreate, e.g. after refactoring into a module), terraform state rm (stop tracking without deleting the real resource), terraform force-unlock (release a stuck lock after a crashed CI job).

A2. terraform plan / apply Internals — Dependency Graph Example

Every plan reconciles three things: config (.tf files, desired state), the state file (last known state), and real infrastructure (queried via provider API refresh). If state says a resource exists but the API says it's gone, Terraform detects drift and plans to recreate it even though config didn't change.

Example — VPC + EKS setup:

```
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}

resource "aws_subnet" "private" {
  vpc_id     = aws_vpc.main.id
  cidr_block = "10.0.1.0/24"
}

resource "aws_iam_role" "eks_role" {
  name               = "eks-cluster-role"
  assume_role_policy = data.aws_iam_policy_document.eks_assume.json
}

resource "aws_eks_cluster" "flightsystem" {
  name     = "flightsystem-1"
  role_arn = aws_iam_role.eks_role.arn
  vpc_config {
    subnet_ids = [aws_subnet.private.id]
  }
}

resource "aws_eks_node_group" "workers" {
  cluster_name  = aws_eks_cluster.flightsystem.name
  node_role_arn = aws_iam_role.node_role.arn
  subnet_ids    = [aws_subnet.private.id]
}
```

Terraform parses every resource.attribute reference and builds a directed acyclic graph (DAG). aws_vpc.main and aws_iam_role.eks_role have no dependency on each other, so they're created in parallel. aws_subnet.private waits for the VPC. aws_eks_cluster.flightsystem waits for both the subnet and the IAM role. aws_eks_node_group.workers waits for the cluster. This is why apply output shows resources completing in batches rather than strictly top to bottom.

depends_on is needed when a relationship exists but no attribute reference captures it — e.g. IAM role propagation being eventually consistent, so the node group must explicitly wait for a policy attachment to fully complete even though it never references that attachment's attributes:

```
resource "aws_eks_node_group" "workers" {
  cluster_name  = aws_eks_cluster.flightsystem.name
  node_role_arn = aws_iam_role.node_role.arn
  depends_on    = [aws_iam_role_policy_attachment.node_policy]
}
```

terraform destroy walks the same graph in reverse — node group destroyed before cluster, cluster before subnet/IAM role, subnet before VPC — which is why manual out-of-order deletion in the console is risky but destroy handles it safely.

A3. Handling Secrets — AWS Secrets Manager + Terraform + RDS

Secrets Manager secret types: 'Credentials for RDS database' (or Redshift/DocumentDB/other AWS database) links the secret directly to the DB instance ARN and enables AWS's built-in automatic rotation Lambda. 'Other type of secret' is a free-form key/value secret for API keys, tokens, or anything not natively integrated — rotation must be built manually. For an RDS-backed app like flightsystem, the RDS-specific type is the right choice since it gives managed rotation out of the box.

Important nuance: state stores every attribute a resource requires, including passwords. Any pattern where Terraform sets a DB password value — whether hardcoded, from a variable, or from random_password — writes that value into terraform.tfstate in plaintext regardless of where the value originated. Reading a secret via a data source doesn't avoid this either, since the value still becomes a tracked resource attribute the moment it's assigned to something like aws_db_instance.password.

The only pattern where Terraform never sees the plaintext at all is letting RDS manage its own master password:

```
resource "aws_db_instance" "flight_db" {
  username                    = "admin"
  manage_master_user_password = true
  # AWS creates & owns the secret in Secrets Manager;
  # Terraform never handles the plaintext value
}
```

The application (e.g. Spring Boot) then retrieves the password at runtime directly from Secrets Manager via an IAM role, never through application.properties or Terraform. For any secret Terraform does have to set, the real control is securing the state backend itself — encrypted S3 (encrypt = true / SSE-KMS), tightly scoped IAM on the state bucket, and never committing .tfstate to version control — rather than trying to avoid hardcoded strings alone.

Appendix B — Additional Commonly Asked Topics

B1. Core Mechanics

terraform refresh

refresh reconciles the state file with real infrastructure without changing config or making any create/update/destroy calls — it just updates state to match what's actually in the cloud. In modern Terraform (0.15+), this reconciliation happens automatically as part of every plan and apply, so running terraform refresh standalone is rarely needed anymore; it still exists for edge cases like inspecting drift without triggering a plan.

terraform validate vs fmt vs plan

validate checks that config is syntactically valid and internally consistent (correct argument names/types, valid references) without contacting any provider API — fast, good for CI on every commit. fmt rewrites files into Terraform's canonical style (indentation, alignment) — purely cosmetic, doesn't check correctness. plan actually

talks to the provider API and state to compute a real diff — the only one of the three that tells you what will actually change.

Provisioners (local-exec, remote-exec)

A provisioner runs a script on a local machine (local-exec) or on the newly created resource itself over SSH/WinRM (remote-exec) as part of resource creation — e.g. running a shell command right after an EC2 instance boots. HashiCorp explicitly documents these as a last resort because they break Terraform's core model: provisioners are imperative, not tracked in state, not idempotent, and if they fail mid-run they can leave a resource in a tainted, hard-to-reason-about state. The recommended alternative is to bake configuration into the image itself (e.g. a pre-built AMI via Packer) or hand off to a dedicated config-management/orchestration tool (Ansible, cloud-init, Kubernetes) rather than scripting inline.

Data sources vs resources

A resource block tells Terraform to create and manage something new — it owns the full lifecycle (create, update, destroy). A data source (data "aws_vpc" "existing" { ... }) only reads information about something that already exists, managed by Terraform or not — Terraform never creates, modifies, or destroys it. Data sources are used to reference existing infrastructure, like looking up an existing VPC ID or the latest AMI, without taking ownership of it.

.terraform.lock.hcl

This file records the exact provider plugin versions (and their checksums) that were selected during init, similar to a package-lock.json. It should be committed to git so every teammate and every CI run installs the identical provider versions, avoiding 'works on my machine' bugs caused by two people silently using different provider versions.

B2. State & Scale

Recovering from a stuck state lock

If a CI pipeline crashes mid-apply, the DynamoDB lock entry can be left behind even though no apply is actually running, blocking every future apply with 'Error acquiring the state lock'. The fix is terraform force-unlock <LOCK_ID> — but only after confirming no apply is genuinely still in progress, since force-unlocking during a real concurrent run can corrupt state.

Partial backend configuration

Instead of hardcoding backend values (bucket name, key, region) directly in the backend "s3" block, you can leave them blank in code and supply them at init time via terraform init -backend-config="bucket=my-bucket" -backend-config="key=envs/dev/terraform.tfstate". This lets the same codebase be initialized against different state locations per environment or per CI pipeline run, without duplicating or hardcoding environment-specific values in the .tf files.

state rm vs destroy

terraform state rm removes a resource from Terraform's state file only — the real resource in AWS is untouched and keeps running, Terraform simply stops tracking/managing it. terraform destroy does the opposite in effect: it actually deletes the real resource in the cloud and then removes it from state. state rm is used when handing a resource off to another team or Terraform config; destroy is used when you actually want the infrastructure gone.

Recovering from a lost state file

If state is lost entirely (and no S3 versioned backup exists), Terraform has no record of what it manages, even though the real infrastructure still exists. Recovery means manually rebuilding state resource by resource using terraform import <resource_address> <real_resource_id> for every existing object, then writing matching resource blocks in config. This is slow and error-prone at scale — which is exactly why versioned remote state (S3 with versioning enabled) is treated as non-negotiable in production, since it lets you simply roll back to the last good version instead of reconstructing state by hand.

B3. Variables & Structure

Variable precedence order

From highest to lowest priority: (1) -var or -var-file flags passed on the CLI, (2) *.auto.tfvars files (alphabetical order if multiple), (3) terraform.tfvars, (4) TF_VAR_ environment variables, (5) the default value inside the variable block itself. A value set higher in this list overrides one set lower.

Sensitive outputs

Marking an output sensitive = true hides its value from standard CLI output (plan/apply logs just show <sensitive>), preventing accidental exposure in terminal history or CI logs. The actual value is still stored in state and can be retrieved deliberately with terraform output -json or terraform output <name> -raw — sensitive only affects default display, not access or state storage.

terraform.tfvars vs *.auto.tfvars

Both are auto-loaded by Terraform without needing a -var-file flag. terraform.tfvars is the single conventional default file. Any file ending in .auto.tfvars (e.g. dev.auto.tfvars, secrets.auto.tfvars) is also auto-loaded, which is useful for splitting variable values into multiple logical files instead of one large terraform.tfvars.

B4. Real-World / Scenario-Based Questions

"Your terraform apply is hanging — how do you debug it?"

Set TF_LOG=DEBUG (or TRACE for maximum detail) before running apply to see exactly which API call Terraform is waiting on. Common causes: a resource waiting on an AWS eventual-consistency delay (e.g. IAM propagation), a provider stuck retrying a rate-limited API call, or a resource whose creation genuinely takes a long time (e.g. an RDS instance or EKS cluster, which can take 10-20 minutes). Cross-check the AWS console/CloudTrail to see if the API call actually succeeded but Terraform is still polling for a status change.

"apply failed halfway through — what state is your infra in?"

Terraform applies changes resource by resource per the dependency graph, and it updates state incrementally as each resource succeeds — so a partial failure leaves you with some resources created and tracked in state, and the failed resource (and anything downstream of it) not yet created. The fix is simply to run terraform plan again to see the current diff, resolve whatever caused the failure (e.g. a naming conflict, quota limit, or permissions issue), and re-run apply — Terraform will only act on what's still missing, not recreate what already succeeded.

"How would you structure Terraform for dev/staging/prod?"

The most common production pattern is separate directories per environment (envs/dev, envs/staging, envs/prod), each with its own backend configuration pointing at a separate state file, all calling shared reusable modules (e.g. a modules/eks-cluster module used identically by every environment with different input variables). This gives full isolation — a mistake in dev can't touch prod state — while keeping the actual infrastructure logic defined once. Workspaces are a lighter-weight alternative for simpler cases, but many teams avoid them for production because it's easy to accidentally run apply against the wrong workspace without realizing it.

"How do you run Terraform in a CI/CD pipeline?"

A typical pattern: on a pull request, CI runs terraform fmt -check, terraform validate, and terraform plan, posting the plan output as a PR comment for review — no infrastructure changes happen yet. On merge to main, a separate pipeline stage runs terraform apply against the same plan, often behind a manual approval gate for production. The backend is always remote (S3 + DynamoDB) so the pipeline and any engineer see the same state, and credentials are scoped to a CI-specific IAM role rather than personal credentials. This maps directly onto the kind of staged pipeline you already built in Jenkins for your flightsystem project — the same plan-then-gate-then-apply structure, just for infrastructure instead of application code.

B5. HashiCorp Ecosystem Awareness

Terraform Cloud / Enterprise

Open-source Terraform is just the CLI — you provide your own remote backend, locking, and CI orchestration (as described above). Terraform Cloud/Enterprise is HashiCorp's managed platform that provides remote state storage

and locking out of the box, remote plan/apply execution (so runs happen in HashiCorp's environment rather than a laptop or self-managed CI runner), a UI for reviewing plans and approving applies, and team/role-based access control — essentially productizing the backend + CI/CD pattern that teams otherwise build themselves with S3, DynamoDB, and Jenkins.

Sentinel / policy-as-code

Sentinel is HashiCorp's policy-as-code framework (used in Terraform Cloud/Enterprise) that lets an organization define rules a plan must pass before it's allowed to apply — for example, 'no S3 bucket may be created without encryption enabled' or 'EC2 instances must be tagged with a cost-center'. It runs automatically between plan and apply, rejecting non-compliant plans before any infrastructure changes happen. Companies use it to enforce security, cost, and compliance standards consistently across every team, rather than relying on manual code review to catch violations.

Appendix C — Terraform Interview Question Bank (Full Answers)

Basic

1. What is Terraform, and what is Infrastructure as Code (IaC)?

Terraform is an open-source tool by HashiCorp that lets you define infrastructure (servers, networks, databases, clusters) as declarative configuration files instead of clicking through a console. IaC is the broader practice this belongs to: describing infrastructure in version-controlled code so it can be reviewed, reused, and provisioned consistently and repeatably, rather than being manually configured and undocumented.

2. What is a Terraform provider?

A provider is a plugin that lets Terraform communicate with a specific platform's API — AWS, Azure, GCP, Kubernetes, GitHub, etc. It translates .tf resource definitions into real API calls and is declared with a provider block, e.g. provider "aws" { region = "ap-northeast-1" }.

3. What is a Terraform resource block?

A resource block is the fundamental unit that tells Terraform to create and manage one real infrastructure object, e.g. resource "aws_instance" "web" { ami = "...", instance_type = "t2.micro" }. Terraform tracks its full lifecycle — create, update in place, or destroy — based on changes to this block.

4. What is a Terraform data source, and how is it different from a resource?

A data source (data "aws_vpc" "existing" { ... }) reads information about something that already exists, without creating, modifying, or destroying it. A resource owns the full lifecycle of an object; a data source only looks up read-only information — useful for referencing existing infrastructure like an existing VPC ID or the latest AMI.

5. What is the Terraform state file, and why is it important?

terraform.tfstate is a JSON file that records every resource Terraform manages, including internal attributes/IDs that never appear in your code. It's Terraform's source of truth for what currently exists, letting it compute an accurate diff on every plan instead of treating every run as a fresh creation.

6. What is the difference between terraform init, plan, and apply?

init sets up the working directory: downloads provider plugins, configures the backend, downloads modules, and creates a lock file. plan compares config, state, and real infrastructure to preview what would change, without touching anything. apply executes those changes for real and updates state to match.

7. What is terraform destroy used for?

It reads the state file, determines everything Terraform currently manages, and deletes those resources in the correct dependency order (reverse of creation order), then updates state to reflect that nothing remains.

8. What is a Terraform variable, and how do you define default values?

A variable (declared with a variable block) is an input parameter that lets the same configuration be reused with different values, e.g. per environment. A default value is set directly in the block:

```
variable "instance_type" {
  type    = string
  default = "t2.micro"
}
```

If no value is supplied via CLI flag, tfvars file, or environment variable, Terraform falls back to this default.

9. What is a Terraform output?

An output block returns a value after apply completes — useful for displaying important information (like a resource endpoint) in the CLI, or for passing data from a child module up to a parent module.

10. What file extension do Terraform configuration files use?

Terraform configuration files use the .tf extension (or .tf.json for JSON-formatted configuration, which is rarely used by hand).

11. What is HCL (HashiCorp Configuration Language)?

HCL is the declarative configuration language Terraform (and other HashiCorp tools) use to define infrastructure. It's designed to be both human-readable and machine-parseable, supporting blocks, arguments, expressions, and references between resources.

12. What is terraform.tfvars used for?

It's a file that assigns actual values to declared variables, instead of hardcoding them or typing them at the CLI every run. Terraform auto-loads terraform.tfvars (and any *.auto.tfvars files) without needing an explicit flag.

13. What is a Terraform provisioner?

A provisioner (local-exec, remote-exec) runs a script either on the local machine or on a newly created resource itself, as part of resource creation — e.g. running a startup command right after an EC2 instance boots. HashiCorp treats provisioners as a last resort since they're imperative, not tracked in state, and not idempotent; baking configuration into an image (e.g. via Packer) or using a dedicated config-management tool is generally preferred.

14. What is terraform fmt and terraform validate used for?

fmt rewrites .tf files into Terraform's canonical style (indentation, alignment) — purely cosmetic. validate checks that configuration is syntactically valid and internally consistent (correct argument names/types, valid references) without contacting any provider API — fast and suitable for every CI run, but it doesn't tell you what will actually change.

15. What is the difference between a local and remote backend?

A local backend stores terraform.tfstate as a plain file on the machine running Terraform — fine for solo experimentation, but unsafe for teams since nobody else sees accurate plans and concurrent applies can corrupt state. A remote backend (e.g. S3) stores state centrally so every engineer and CI pipeline read/write the same source of truth, and can be paired with locking (e.g. DynamoDB) to prevent concurrent-apply corruption.

Intermediate / Scenario

16. Why should Terraform state be stored remotely (e.g., S3) instead of locally?

Local state means only one person's machine has the record of what's deployed — nobody else gets accurate plans, a lost laptop means losing Terraform's only memory of managed infrastructure, and there's no way to safely coordinate applies across a team or CI pipeline. A remote backend like S3 centralizes state so everyone reads/writes the same file, and (with versioning enabled) provides a rollback path if state ever becomes corrupted.

17. What is state locking, and why is DynamoDB commonly paired with an S3 backend?

State locking prevents two applies from running against the same state at the same time. When paired with S3, Terraform uses a DynamoDB table to hold a lock entry during any apply; if a second apply attempts to start, it's blocked with 'Error acquiring the state lock' until the first one finishes and releases the lock — preventing simultaneous writes from corrupting state or double-provisioning resources. (Newer Terraform versions also support native S3-based locking without DynamoDB.)

18. What is a Terraform module, and why use modules?

A module is a reusable, packaged group of resources — a folder of .tf files callable multiple times with different inputs instead of duplicating code. Modules standardize how something like a VPC or EKS cluster is built, so the same tested logic is reused across dev/staging/prod (or across teams) by simply varying input variables, rather than copy-pasting and risking drift between copies.

19. How do you pass variables into a module?

By declaring input variables inside the module (variable blocks in the module's own .tf files) and then supplying values as arguments when calling the module from the parent configuration:

```
module "vpc" {
  source   = "./modules/vpc"
  cidr     = "10.0.0.0/16"
  env_name = var.environment
}
```

20. What is the difference between count and for_each in Terraform?

count creates multiple instances of a resource from a number, addressed by index (resource.name[0]). for_each creates instances from a map or set of strings, addressed by a stable key (resource.name["key"]). The practical difference shows up when the list changes: removing a middle item under count shifts every later index, causing Terraform to destroy/recreate resources that didn't actually change; for_each avoids this since each instance is tied to its own key regardless of order.

21. What is a dynamic block used for?

A dynamic block lets you generate repeated nested configuration blocks (like multiple ingress rules inside a security group) programmatically from a list or map, instead of writing each nested block out by hand — useful when the number of nested blocks varies per environment or input.

22. What is the Terraform dependency graph, and how does Terraform determine resource creation order?

Before creating anything, Terraform builds a directed acyclic graph (DAG) from every resource-to-resource attribute reference in the config. It walks this graph to decide execution order — independent resources are created in parallel, dependent resources strictly in sequence. Where a dependency exists but isn't captured by any attribute reference (e.g. IAM propagation timing), depends_on forces the ordering explicitly.

23. What is terraform import used for?

import brings an existing resource — created manually, outside Terraform, or after state loss — under Terraform's management by linking it to a resource block in state, without recreating it. You still need a matching resource block written in config; import only populates state (though newer Terraform versions can also generate matching configuration alongside import).

24. What is a lifecycle block, and what does prevent_destroy do?

A lifecycle block customizes how Terraform manages a specific resource's lifecycle, e.g.:

```
resource "aws_db_instance" "prod" {
  # ...
  lifecycle {
    prevent_destroy = true
  }
}
```

prevent_destroy = true causes Terraform to error out and refuse to proceed if a plan/apply/destroy would delete that resource — a safety guard for critical resources like a production database, preventing accidental destruction from a careless apply or destroy.

25. How do you manage multiple environments (dev/staging/prod) cleanly in Terraform (workspaces vs directory structure)?

The most common production pattern is separate directories per environment (envs/dev, envs/staging, envs/prod), each with its own backend config pointing at a separate state file, all calling shared reusable modules with different input variables — giving full isolation so a mistake in dev can't touch prod. Workspaces are a lighter alternative that share the same code and backend but keep separate state per workspace name; many teams avoid them for production because it's easy to run apply against the wrong workspace without noticing.

26. What is a Terraform workspace, and what are its limitations for environment separation?

A workspace lets you maintain multiple isolated state files from the same configuration directory (terraform workspace new dev). The limitation is that all workspaces share identical code and backend configuration — there's no way to give dev and prod different variable defaults, different provider credentials, or different approval gates purely through workspaces, and it's easy to forget which workspace is currently selected before running apply, risking an accidental production change.

27. How would you structure a Terraform repository for a large enterprise with 20+ AWS accounts?

A common pattern: a central repo of versioned, tested modules (e.g. published via a private module registry or tagged Git releases) covering common building blocks (VPC, EKS, RDS, IAM baseline). Each AWS account/team then has its own thin configuration (often in a separate repo or directory) that calls specific module versions with account-specific variables and its own isolated remote state/backend. This keeps blast radius contained per account, allows different teams to upgrade module versions independently, and avoids one giant shared state file across the whole organization.

28. How do you avoid hardcoding secrets in Terraform code?

Reference secrets from a secrets manager (AWS Secrets Manager, SSM Parameter Store) via a data source at apply time rather than writing plaintext values into .tf files, and mark sensitive variables/outputs as sensitive = true to keep them out of CLI logs. Note this reduces exposure in code and logs, but the value can still end up in state if a resource requires it as a plaintext argument — for cases like an RDS master password, using manage_master_user_password = true lets AWS own the secret entirely so Terraform never handles the plaintext at all. Securing the state backend itself (encryption, tight IAM) remains the real control regardless.

29. What happens if two engineers run terraform apply simultaneously without state locking?

Both read the same starting state, compute independent plans, and begin making API calls at the same time. Whichever apply finishes last overwrites the other's state changes, causing drift — potential duplicate resources, orphaned resources Terraform no longer tracks correctly, or a state file that no longer matches real infrastructure. This is exactly what remote backend locking (e.g. S3 + DynamoDB) is designed to prevent.

30. What is drift detection in Terraform, and how do you handle configuration drift?

Drift is when real infrastructure no longer matches what's recorded in state — typically caused by manual changes made outside Terraform (e.g. someone editing a security group rule in the console). Terraform detects this during the refresh step of plan, showing the discrepancy as a proposed change. It's handled either by running apply to force infrastructure back to match config, or by running terraform import / updating config to intentionally adopt the manual change if it should be kept.

31. How do you upgrade a Terraform provider version safely in a live environment?

Update the version constraint in required_providers, run terraform init -upgrade to pull the new version, then run terraform plan and carefully review the diff — provider upgrades can introduce new required arguments, renamed attributes, or behavior changes that show up as unexpected planned changes even though your config didn't change. Test in a non-production environment first, and read the provider's changelog for breaking changes before applying to production.

32. What is the difference between terraform plan -out and just running terraform plan?

A plain terraform plan only prints the proposed diff to the console — it isn't guaranteed to be the exact plan that gets applied if anything changes (config, state, or real infrastructure) between plan and apply. terraform plan -out=tfplan saves the computed plan to a file, and terraform apply tfplan applies exactly that saved plan with no re-computation — this is the safer pattern for CI/CD, ensuring what was reviewed is exactly what gets applied.

33. How would you break a single large Terraform state file into smaller, isolated state files?

Split the configuration by logical boundary (e.g. networking, EKS cluster, RDS, IAM) into separate directories/root modules, each with its own backend config and state file, then use terraform state mv to move existing resources out of the old state into the new one without destroying/recreating them. Cross-references between the split states are then handled via remote state data sources rather than direct in-config references.

34. What is a remote state data source, and how do you reference outputs from another Terraform state?

A terraform_remote_state data source lets one Terraform configuration read the outputs of another configuration's state file, enabling separate state files to still share information:

```
data "terraform_remote_state" "network" {
  backend = "s3"
  config = {
    bucket = "my-company-terraform-state"
    key    = "network/terraform.tfstate"
    region = "ap-northeast-1"
  }
}

# usage: data.terraform_remote_state.network.outputs.vpc_id
```

35. How would you implement a CI/CD pipeline for Terraform with manual approval before production apply?

On a pull request: run terraform fmt -check, validate, and plan -out=tfplan, then post the plan output as a PR comment for review — no changes happen yet. On merge to main, a pipeline stage re-runs plan (or reuses the saved plan artifact) and pauses at a manual approval gate before running terraform apply tfplan against production. The backend stays remote (S3 + DynamoDB) throughout so pipeline runs and engineers always see the same state, and the pipeline uses a scoped CI-specific IAM role rather than personal credentials.

Troubleshooting

36. terraform apply fails midway through creating resources. What is the state of your infrastructure, and what's your next step?

Terraform applies resources per the dependency graph and updates state incrementally as each one succeeds, so a partial failure leaves some resources created and correctly tracked in state, while the failed resource (and anything downstream of it) is not yet created. The next step is simply to run terraform plan again to see the current diff, fix whatever caused the failure (naming conflict, quota limit, permissions), then re-run apply — Terraform only acts on what's still missing, not what already succeeded.

37. Terraform state shows a resource exists, but it was deleted manually in the AWS console. How do you resolve this drift?

Run terraform plan — Terraform's refresh step will detect that the resource no longer exists in real infrastructure and propose recreating it. If the resource should still exist, apply to recreate it per config. If the manual deletion was intentional, instead remove the resource from state with terraform state rm (or delete its block from config) so Terraform stops expecting it.

38. terraform plan shows unexpected changes/destruction of resources you didn't modify. How do you investigate?

Common causes: a provider version upgrade changed a default/computed attribute, someone else applied a change to shared state, a data source is now returning a different value (e.g. 'latest AMI' resolving to a new image), or count/for_each index shifting due to a list reordering. Check git blame/history on the config, check state history if versioned, review the exact attribute Terraform says will change, and check TF_LOG=DEBUG output if the cause isn't obvious from the plan diff alone.

39. A state lock is stuck (e.g., a previous apply crashed) and blocking all new runs. How do you safely resolve it?

First confirm no apply is genuinely still running anywhere (check CI pipeline status, ask teammates). Once confirmed, run terraform force-unlock <LOCK_ID> using the lock ID shown in the error message to manually release the DynamoDB lock entry. Force-unlocking while a real apply is still in progress can corrupt state, so this step should never be taken without first ruling that out.

40. Two team members' local Terraform code has diverged from what's actually deployed. How do you reconcile this?

Run terraform plan against the current (correct) remote state to see the actual diff between the deployed infrastructure and each person's local code. Whoever's code doesn't match should pull the latest config from version control (assuming Terraform code is properly committed to git, as it should always be) rather than trying to manually reconcile — remote state plus git as the single source of truth for both code and state avoids this entirely going forward.

41. A terraform destroy was accidentally run against production. What could have prevented this, and how do you recover?

Prevention: a lifecycle { prevent_destroy = true } block on critical resources, requiring manual approval gates in CI/CD before any apply or destroy reaches production, and restricting who/what has credentials capable of running destroy against the production backend at all. Recovery depends on whether state and backups exist: if the S3 backend has versioning enabled, you can inspect state history, and for resources like RDS, restoring from the most recent automated snapshot is often the only path back; for stateless resources, terraform apply from the last known-good config can recreate them, though data loss on stateful resources (databases, disks) may be unavoidable without a snapshot.

42. Provider version upgrade broke existing resource definitions with deprecated arguments. How do you fix this without downtime?

Read the provider's changelog/migration guide for the specific breaking change, update the deprecated arguments to their new equivalents in config, then run terraform plan carefully — a well-handled provider migration should show no actual infrastructure changes (in-place attribute renames only), not a destroy/recreate. If the diff proposes

replacing a resource, check if the provider offers a moved block or terraform state mv path to migrate state without triggering real infrastructure changes, testing the exact upgrade path in a non-production environment first.

43. Terraform is extremely slow to plan because of a massive single state file. How would you address this?

Split the monolithic state into smaller state files along logical boundaries (networking, compute, database, etc., as in question 33), using terraform state mv to migrate existing resources without recreating them, and terraform_remote_state data sources to share values across the split configurations. This reduces how much state Terraform has to refresh and diff on every plan, and limits blast radius so a change in one area doesn't require planning the entire organization's infrastructure.

44. Sensitive values are appearing in Terraform plan output/logs despite being marked sensitive = true. Why, and how do you truly protect them?

sensitive = true only masks the value in Terraform's own CLI output (plan/apply summaries show <sensitive> instead of the real value) — it does not encrypt the value in the state file, and it doesn't stop the value from appearing in provider-level debug logs, CI system logs that capture raw output, or terraform output -json (which still returns the real value on request). Real protection means encrypting the state backend at rest (S3 SSE-KMS), tightly restricting IAM access to the state bucket, avoiding TF_LOG=DEBUG/TRACE in shared CI logs where secrets might be echoed, and where possible avoiding having Terraform handle the plaintext at all (e.g. manage_master_user_password = true for RDS).

45. A module you depend on (from a Git source) had a breaking change pushed upstream, breaking your pipeline. How do you prevent this going forward (version pinning)?

Pin the module source to a specific tag or commit SHA rather than a moving branch reference, e.g.:

```
module "vpc" {
  source = "git::https://github.com/org/terraform-modules.git//vpc?ref=v1.4.0"
}
```

Referencing ?ref=v1.4.0 (a tagged release) instead of ?ref=main means upstream changes never affect your configuration until you deliberately bump the ref yourself, after reviewing the new version's changelog and testing it in a non-production environment first.